

第一章 勾配消失問題

・勾配消失問題

誤差逆伝播法が下位層に進んでいくに連れて、勾配がどんどん緩やかになっていく。そのため、勾配降下法による、更新では下位層のパラメータはほとんど変わらず、訓練は最適値に収束しなくなる。

・活性化関数: シグモイド関数

0 ~ 1の間を緩やかに変化する関数で、ステップ関数ではON/OFFしかない状態に対し、信号の強弱を伝えられるようになり、予想ニューラルネットワーク普及のきっかけとなった。

・課題

大きな値では出力の変化が微小なため、勾配消失問題を引き起こす事があった。

・勾配消失の解決法

→活性化関数の選択

→重みの初期値設定

→バッチ正規化

・活性化関数: ReLU関数

今最も使われている活性化関数

勾配消失問題の回避とスパース化に貢献することで良い成果をもたらしている。

・重みの初期値設定-Xavier

Xavierの初期値（※）を設定する際の活性化関数

→ReLU関数

→シグモイド（ロジスティック）関数

→双曲線正接関数

※前層のノードの個数を n とした場合、 $1/\sqrt{n}$ の標準偏差を持つ分布を使うこと

```
import numpy as np
node_num = 100
w = np.random.randn(node_num, node_num) / np.sqrt(node_num)
```

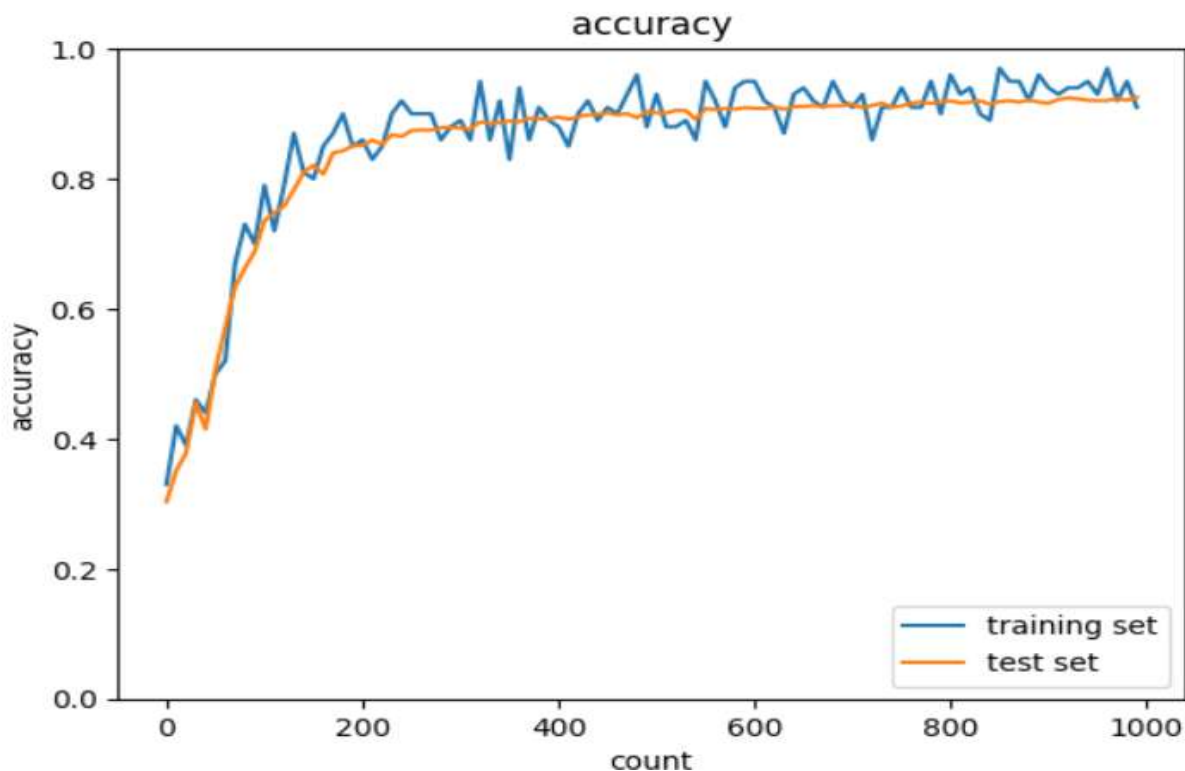
・重みの初期値設定-He

Heの初期値を設定する際の活性化関数

→初期値の設定方法・・・重みの要素を、前の層のノード数の平方根で除算した値に対し $\sqrt{2}$ をかけ合わせた値

実践演習<2_1_network_modified.ipynb>

Generation: 1000. 正答率(トレーニング) = 0.91
: 1000. 正答率(テスト) = 0.9256



まとめ

活性化関数にReLUを使う場合は「Heの初期値」、sigmoidやtanhなどのS字カーブの時は「Xavierの初期値」を使う。→ベストプラクティス

・ バッチ正規化・・・ミニバッチ単位で、入力値のデータの偏りを抑制する手法
使いどころ：活性化関数に値を渡す前後に、バッチ正規化の処理を孕んだ層加える
→バッチ正規化層への入力値は $\mathbf{u}^{(l)} = \mathbf{w}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$ または $\mathbf{z}$

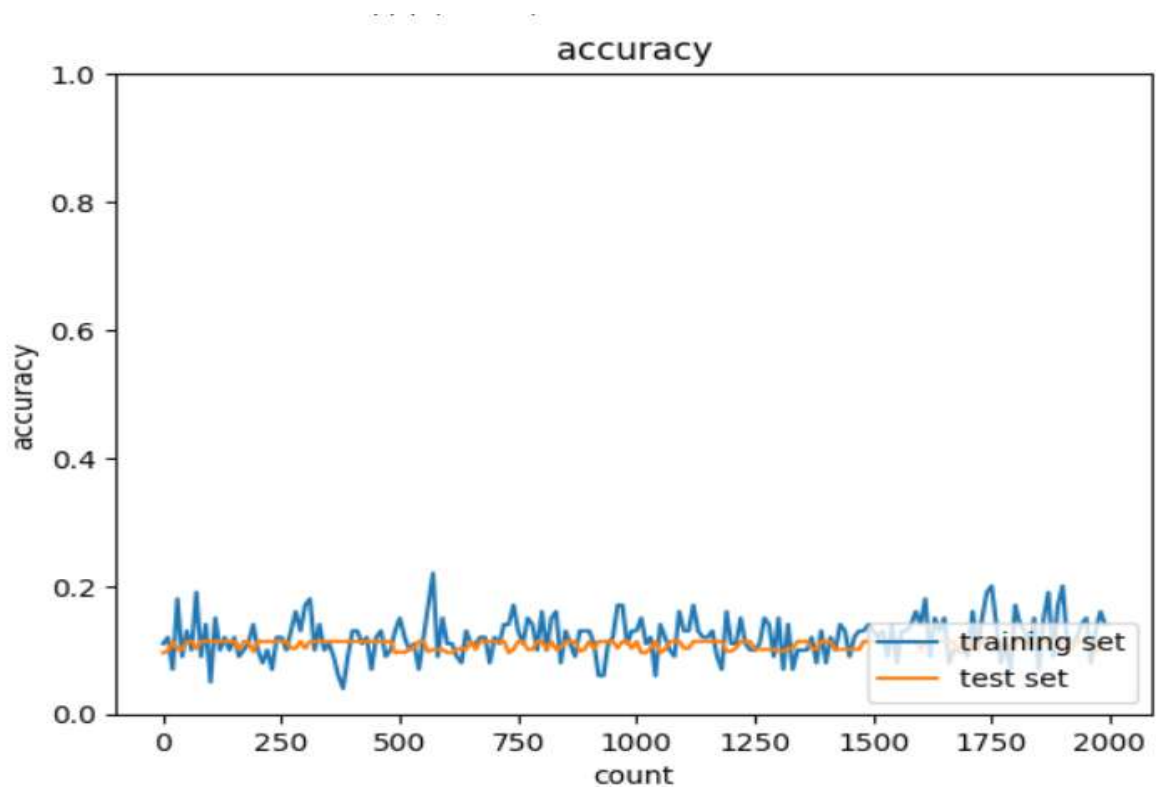
- ・ 中間層の重みが安定化する。→学習がスムーズに進む
- ・ 学習データのバラツキを抑える→過学習を抑えることができる。

・ バッチ正規化の数学的記述

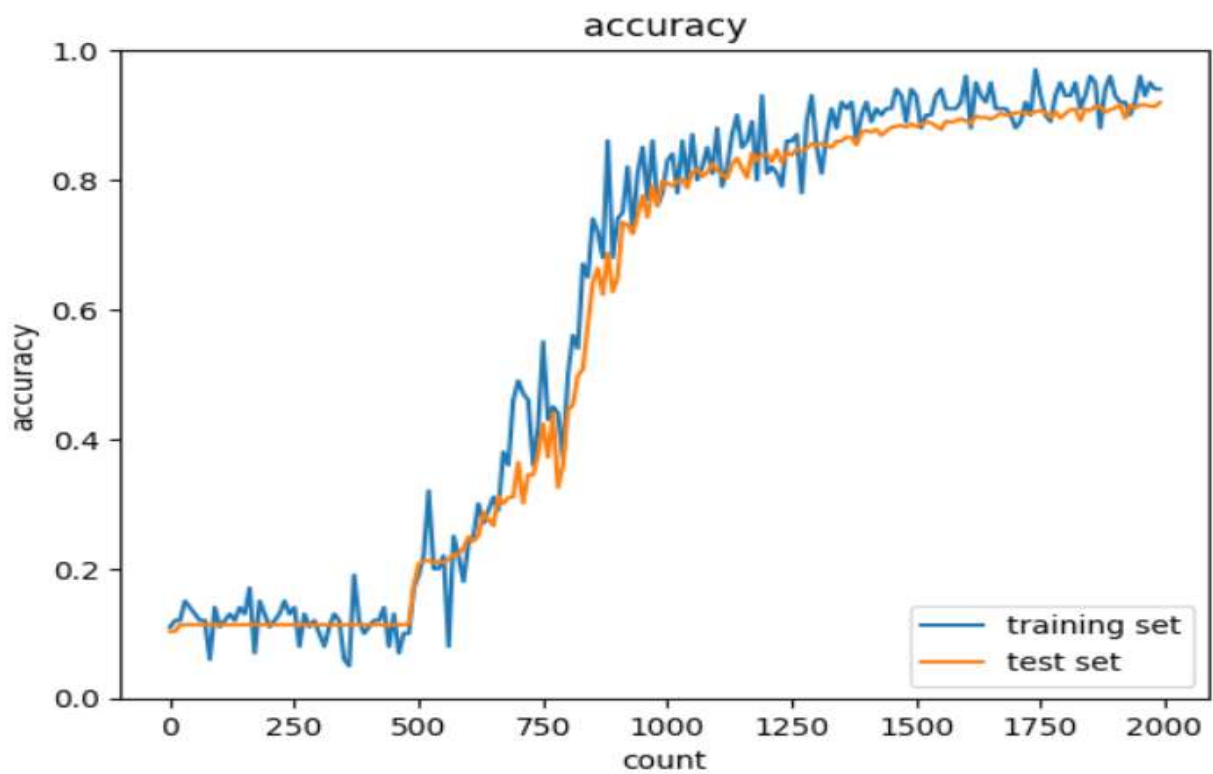
ミニバッチの平均	→	1. $\mu_t = \frac{1}{N_t} \sum_{i=1}^{N_t} x_{ni}$
ミニバッチの分散	→	2. $\sigma_t^2 = \frac{1}{N_t} \sum_{i=1}^{N_t} (x_{ni} - \mu_t)^2$
ミニバッチの正規化	→	3. $\hat{x}_{ni} = \frac{x_{ni} - \mu_t}{\sqrt{\sigma_t^2 + \theta}}$
変倍・移動	→	4. $y_{ni} = \gamma \hat{x}_{ni} + \beta$

実践演習 <2_2_2_vanishing_gradient_modified.ipynb>

- sigmoid - gauss

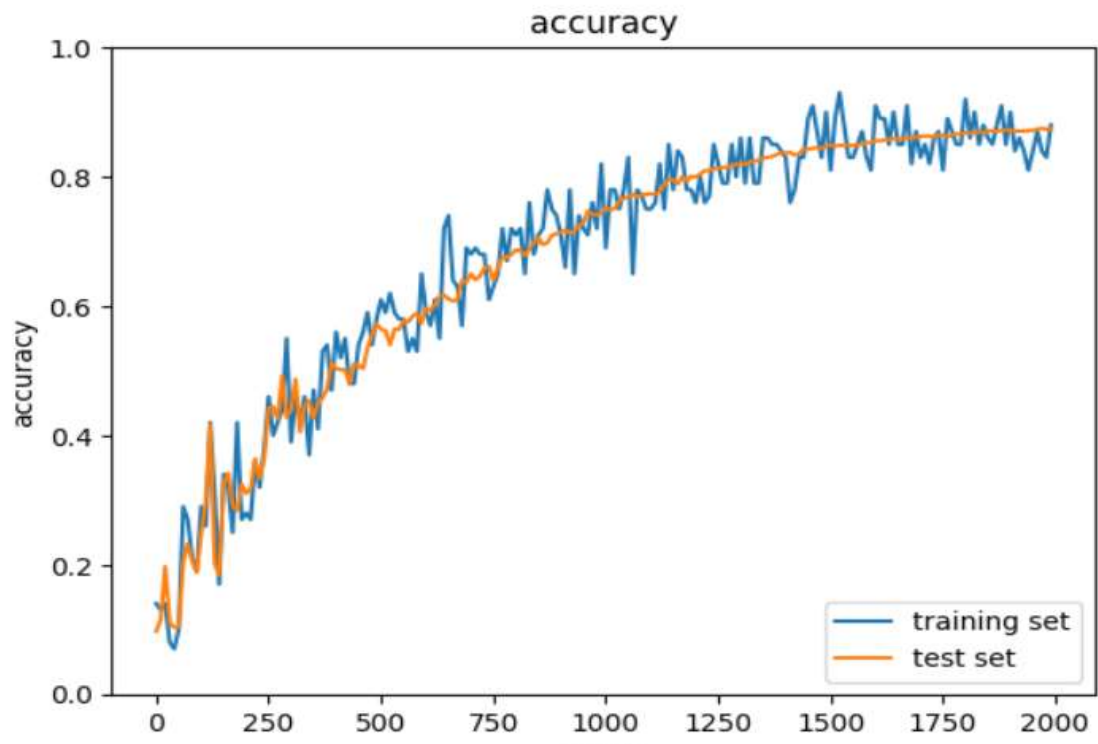


- ReLU - gauss

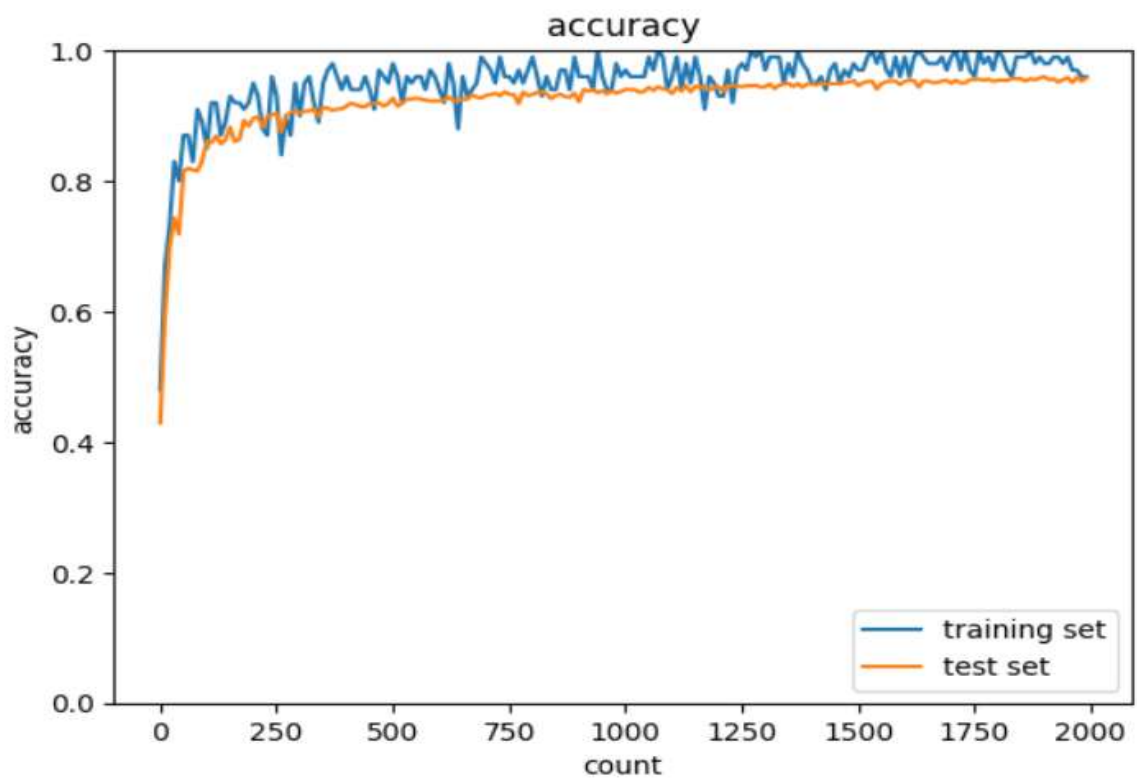


• sigmoid - Xavier

..

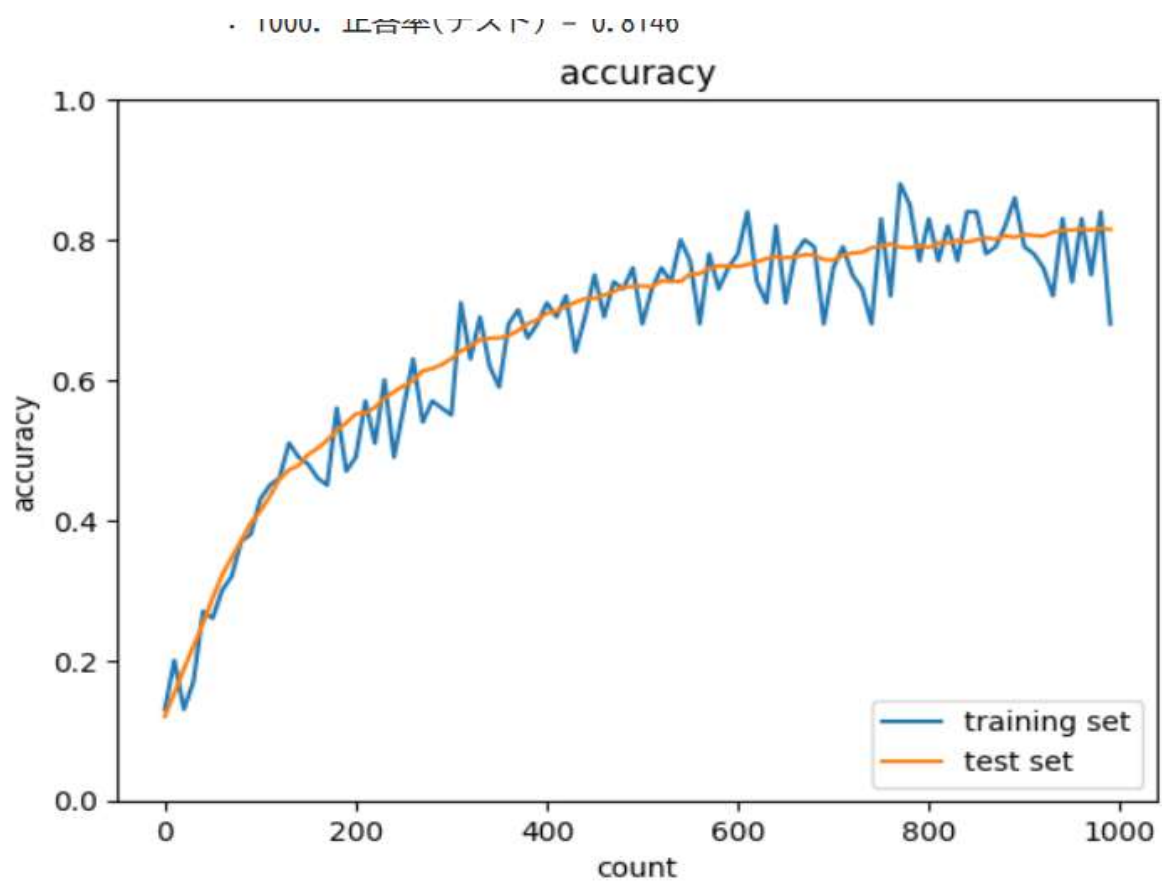


• ReLU - He



実践演習 <2_3_batch_normalization.ipynb>

- batch normalization



第二章 学習率最適化手法

- ・ 学習率の復習

学習率の値が大きい場合・・・最適値にいつまでもたどりつかずに発散してしまう

学習率の値が小さい場合・・・発散することはないが、小さすぎると収束するまでに時間がかかってしまう。

大域居所最適値に収束しづらくなる。

- ・ 学習率の決め方とは

- ・ 初期の学習率設定法の指針

→初期の学習率を大きく設定し、徐々に学習率を小さくしていく

→パラメータごとに学習率を可変させる

→学習率最適化手法を利用して学習率を最適化

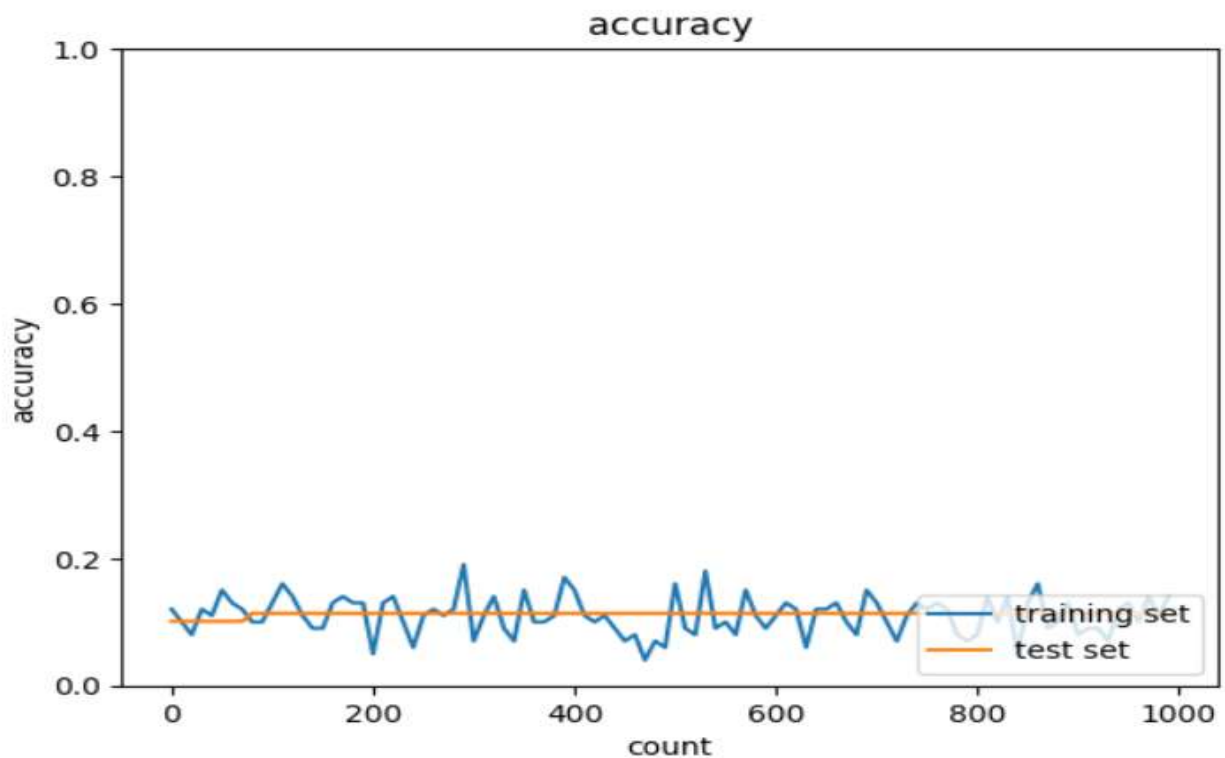
- ・ SGD

データを少しずつ見ながら、最適なパラメータを探す方法

速くて、大規模データに向いていて、ディープラーニングで必須の手法

少しランダムなので、最適解にたどり着きやすい

実践演習 <2_4_optimizer_after.ipynb>



- ・モメンタム

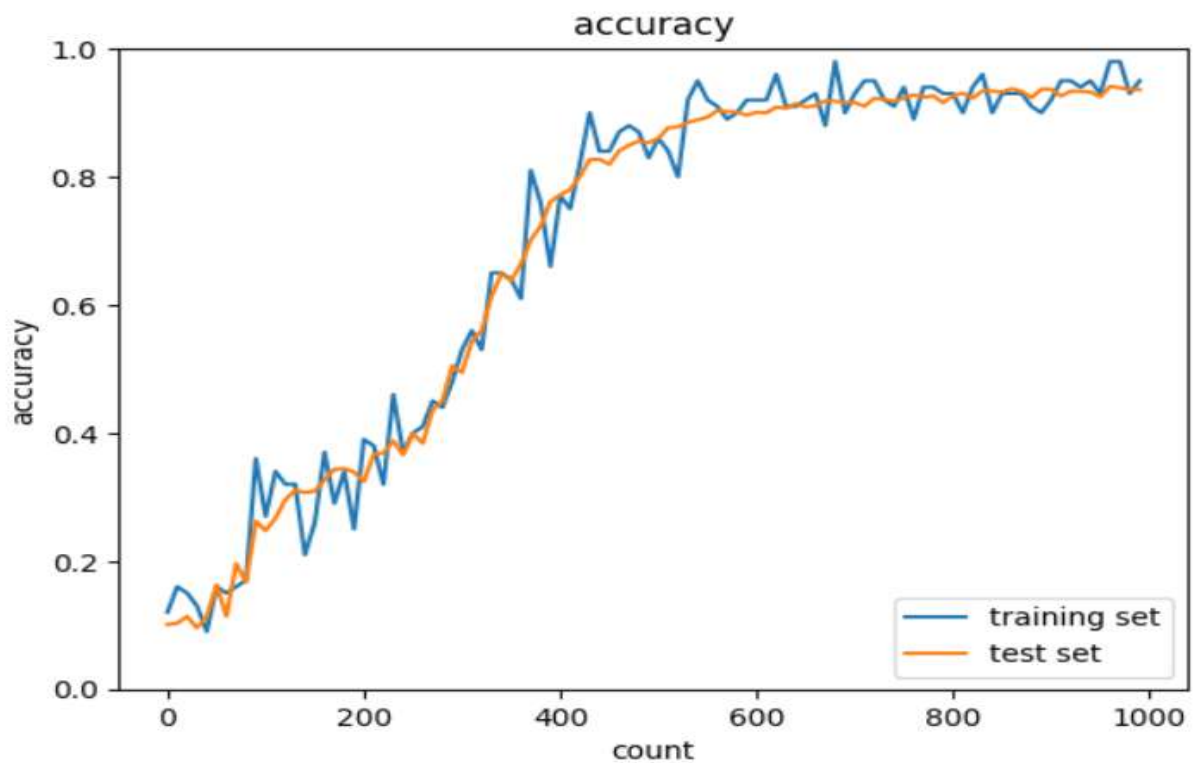
誤差をパラメータで微分したものと学習率の積を減算した後、現在の重みに前回の重みを減算した値と慣性の積を加算する

- ・モメンタムのメリット

→局所的最適解にはならず、大域的最適解となる。

→谷間についてから最も低い位置(最適値)にいくまでの時間が早い。

実践演習<2_4_optimizer_after.ipynb>

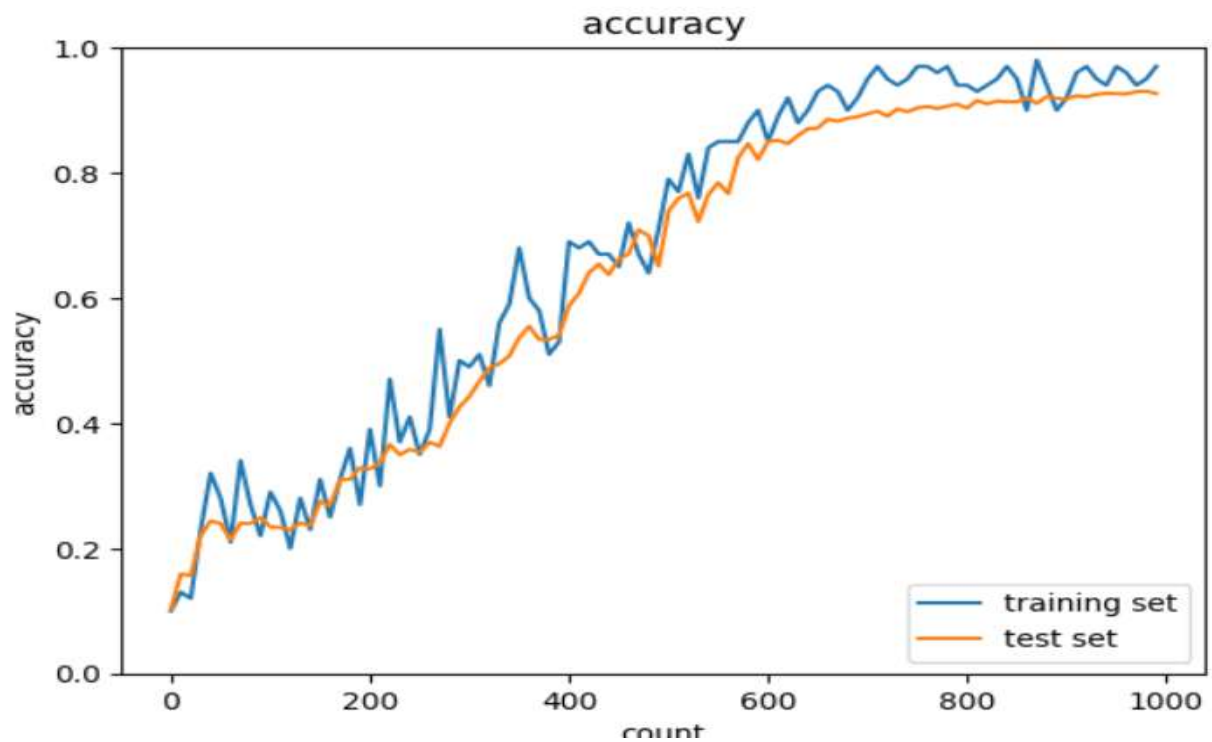


AdaGrad・・・誤差をパラメータで微分したものと再定義した学習率の積を減算する

AdaGradのメリット・・・勾配の緩やかな斜面に対して、最適値に近づける

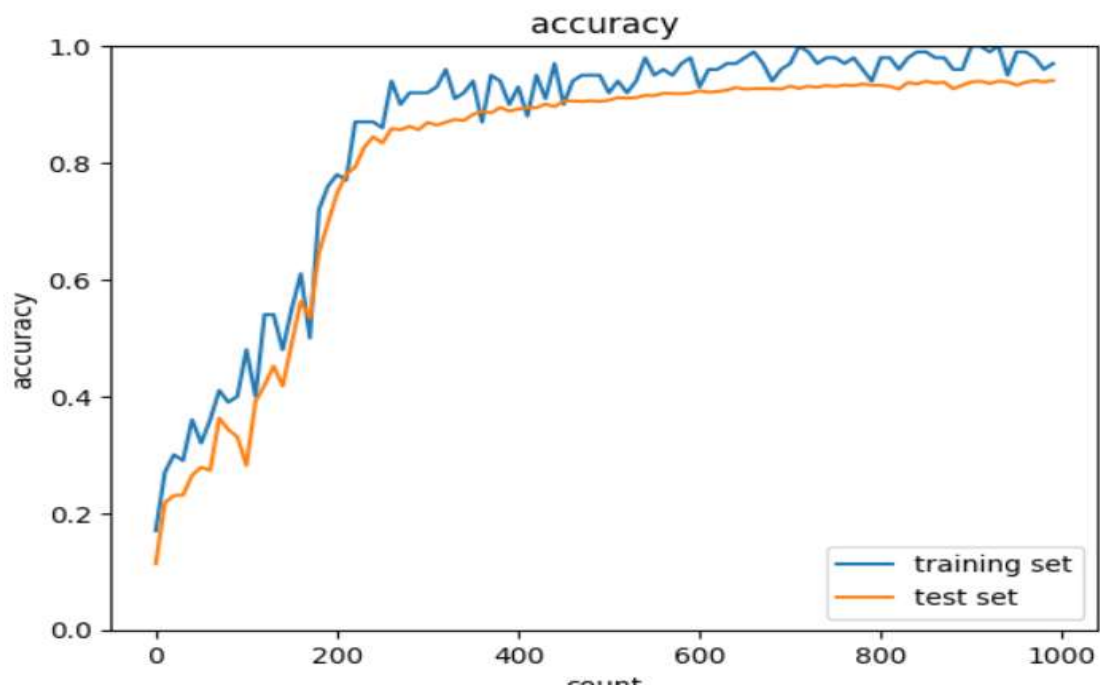
課題・・・学習率が徐々に小さくなるので、鞍点問題を引き起こす事があった。

実践演習 <2_4_optimizer_after.ipynb>



- ・ RMSProp・・・誤差をパラメータで微分したものと再定義した学習率の積を減算する
- ・ RMSPropのメリット・・・局所的最適解にはならず、大域的最適解となる。
ハイパーパラメータの調整が必要な場合が少ない。

実践演習 <2_4_optimizer_after.ipynb>

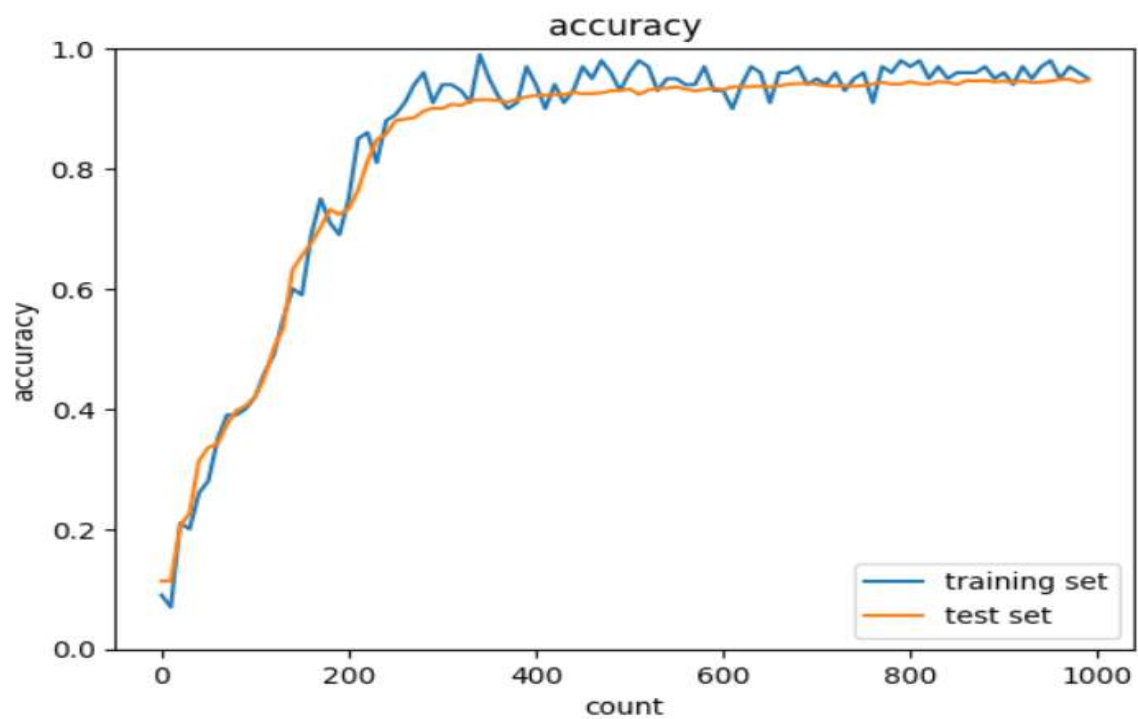


Adam・・・モメンタムの、過去の勾配の指数関数的減衰平均

RMSPropの、過去の勾配の2乗の指数関数的減衰平均

Adamのメリット・・・モメンタムおよびRMSPropのメリットを孕んだアルゴリズム

実践演習 <2_4_optimizer_after.ipynb>



第三章 過学習

- ・ 過学習・・・テスト誤差と訓練誤差とで学習曲線が乖離すること

特定の訓練サンプルに対して、特化して学習する

- ・ 原因・・・パラメータの数が多い

→結果：ネットワークの自由度(層数、ノード数、パラメータの値etc...)が高い

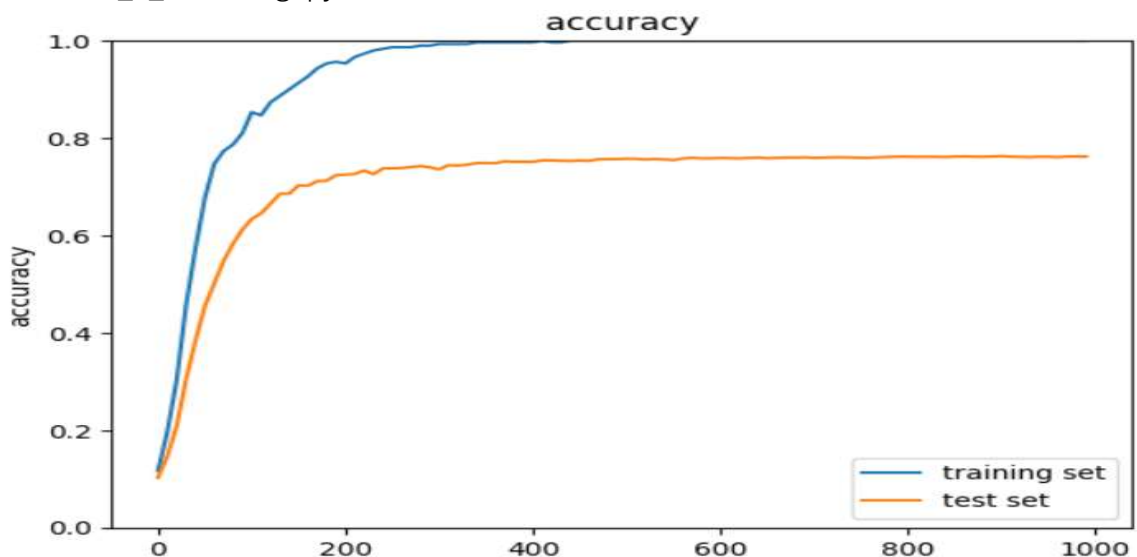
- ・ 過学習の原因・・・重みが大きい値をとることで、過学習が発生することがある。

→学習させていくと、重みにばらつきが発生する。重みが大きい値は、学習において重要な値であり、重みが大きいと過学習が起こる

- ・ 過学習の解決策・・・誤差に対して、正則化項を加算することで、重みを抑制する。

→過学習がおこりそうな重みの大きさ以下で重みをコントロールし、かつ重みの大きさにばらつきを出す必要がある。

<実践演習：2_5_overfitting.ipynb>



- ・ 正則化・・・ネットワークの自由度(層数、ノード数、パラメータの値etc...)を制約すること

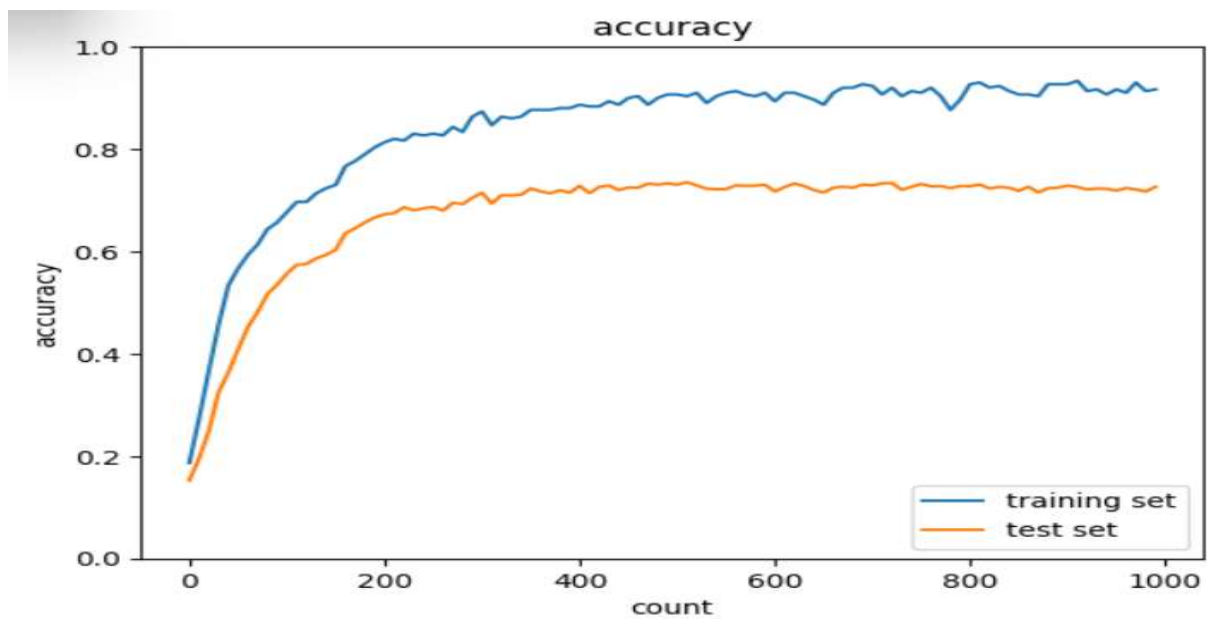
→正則化手法を利用して過学習を抑制する

L1正則化（ラッソ回帰）・・・ $p1$ ノルム（絶対値の和）を加算

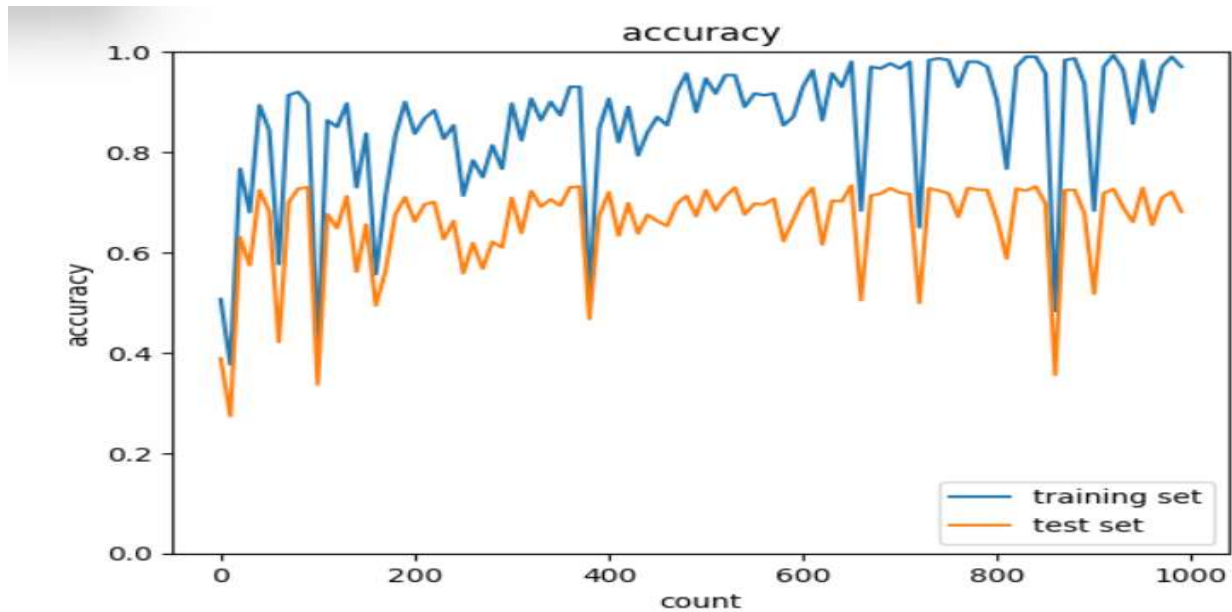
L2正則化（リッジ回帰）・・・ $p2$ ノルム（各要素の2乗和）を加算

<実践演習：2_5_overfitting.ipynb>

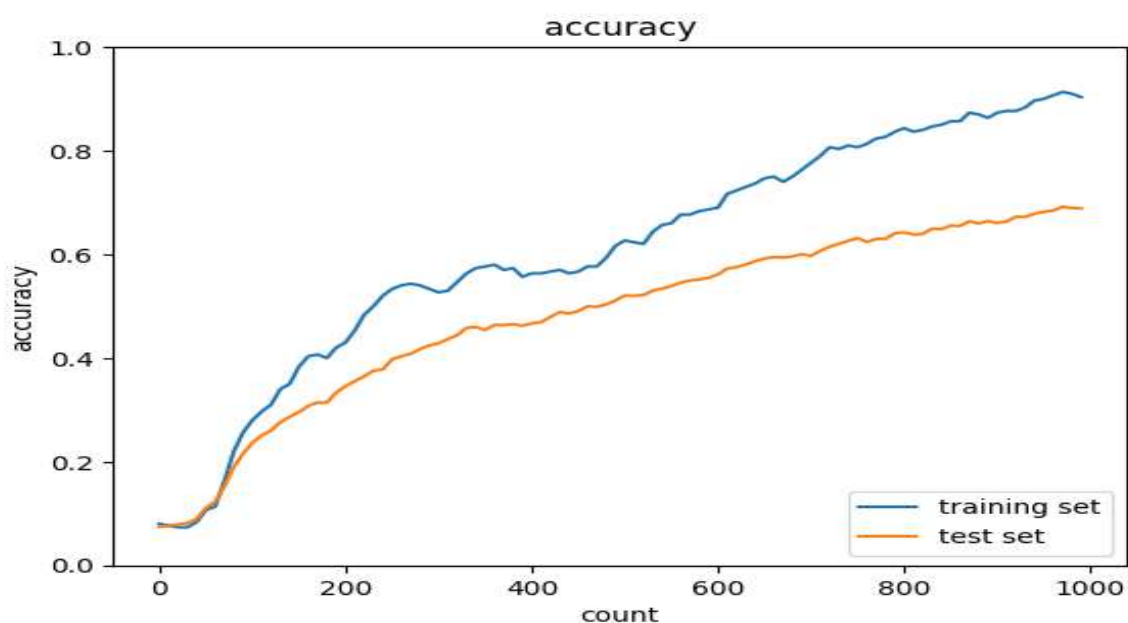
・ L2正則化



L1正則化



- ・ドロップアウト・・・ランダムにノードを削除して学習させること
 - ・メリット・・・データ量を変化させずに、異なるモデルを学習させていると解釈できる
- <実践演習：2_5_overfitting.ipynb>



■ 陽的正則化と陰的正則化

	陽的正則化	陰的正則化
概要	実装モデルの損失関数に正則化項を追加する方法	実装モデルの各種パラメータの調整が正則化の役割を果たす方法
代表的な手法	L1正則化 L2正則化	早期終了(アーリーストッピング) ドロップアウト

- ・陰的正則化における調整パラメータ
 - エポック数：訓練データ全てを入力として、学習を行う回数
 - エポック数の値によっては、過学習になるリスクが高まる
 - バッチサイズ：ミニバッチ学習における、1つのバッチのデータサイズ
 - バッチサイズが大きさによっては、大域最適解に収束しにくくなる。
 - 学習率：モデルの重みを更新する際のステップの大きさ
 - 学習率の大きさによっては、大域最適解に収束しにくくなる。

- ・エポック数の正則化

早期終了(アーリーストッピング)

→検証データセットの性能が改善しなくなったときに学習を終了するテクニック

→これにより、過学習を防ぎ、モデルの汎化性能を向上

- ・早期終了の終了条件

→どのタイミングで学習を終了するかは、モデルの性能と検証データの誤差を確認

しながら決定。一般的に、誤差の値が一定の回数改善しない場合、学習を終了

- ・バッチサイズの大きさによる影響

	訓練速度	メモリ使用量	ノイズの影響	極小値の性質
バッチサイズ 大	速くなる	増加する	受けづらい	幅が狭く 鋭い
バッチサイズ 小	遅くなる	減少する	受けやすい	幅が広く 平ら

- ・バッチサイズの決め方

→最適なバッチサイズは、タスクやデータに依存するため、試行錯誤が必要となる。

→一般的に 2^n の値に設定する。

- ・学習率の大きさによる影響

- ・学習率：大

→大域最適解に収束せず発散する可能性が増加する。

→局所最適解に陥る可能性が低下する

- ・学習率：小

→大域最適解に収束する可能性が増加する

→局所最適解に陥る可能性が増加する

- ・学習率の決め方

→最適なバッチサイズは、タスクやデータに依存するため、試行錯誤が必要となる。

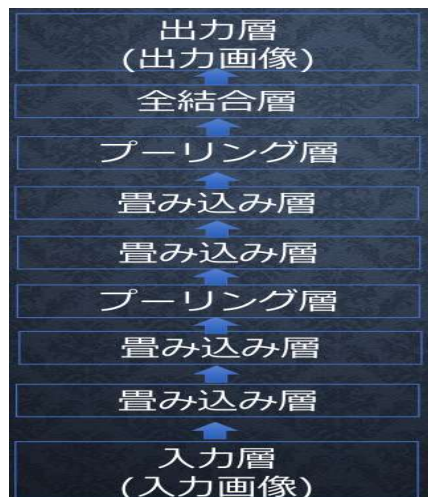
→学習率のスケジューリングや最適化アルゴリズムなどを用いて、自動調整する手法もある。

第4章 畳み込みニューラルネットワークの概念

- ・ 畳み込み層・・・畳み込み層では画像の場合、縦、横、チャンネルの3次元のデータをそのまま学習し、次に伝えることができる。

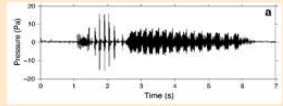
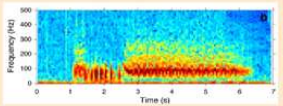
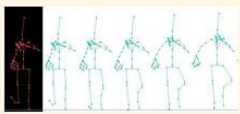
→結論:3次元の空間情報も学習できるような層が畳み込み層である。

- ・ CNNの構造図（例）

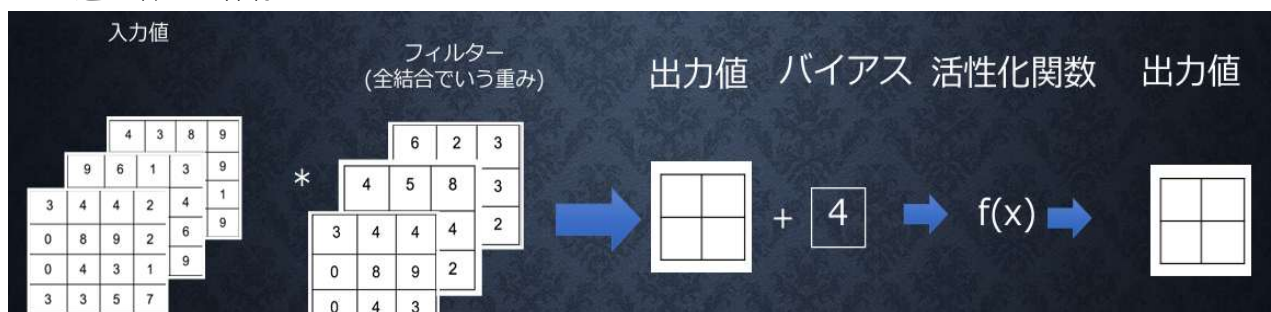


- ・ CNNで扱えるデータの種類

CNNでは次元間で繋がりのあるデータを扱える

	1次元	2次元	3次元
単一チャンネル	音声 [時刻, 強度] 	フーリエ変換した音声 [時刻, 周波数, 強度] 	CTスキャン画像 [x, y, z, 強度] 
複数チャンネル	アニメのスケルトン [時刻, (腕の値, 膝の値 ...)] 	カラー画像 [x, y, (R, G, B)] 	動画 [時刻, x, y, (R, G, B)] 

- ・ 畳込み層の全体像



- ・畳み込み演算・・・画像処理でいうところの「フィルター演算」
 - 入力データに対して、フィルターを適用
 - 入力データに対して、フィルターのウィンドウを一定の間隔でスライドさせながら適用
 - バイアス項の加算は、フィルター適用後のデータに対して行われる。
- ・パディング・・・畳み込み層の処理を行う前に、入力データの周囲に固定のデータ（例えば0など）を埋めること
- ・パディングを使用する理由→出力サイズを調整するため。
- ・ストライド・・・フィルターを適用する一の間隔をストライドという。ストライドを大きくすると、出力サイズは小さくなる。

<実践演習：2_6_simple_convolution_network_after.ipynb>

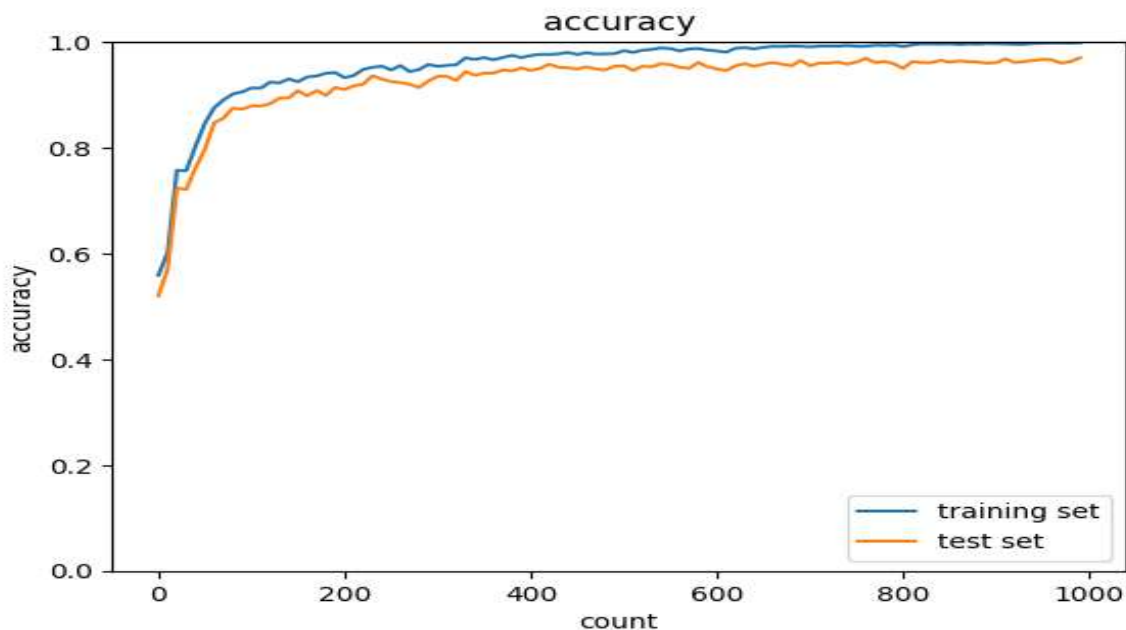
```
[6] 0 秒
✓
# im2colの処理確認
input_data = np.random.rand(2, 1, 4, 4)*100//1 # number, channel, height, widthを表す
print('===== input_data =====\n', input_data)
print('=====')
filter_h = 3
filter_w = 3
stride = 1
pad = 0
col = im2col(input_data, filter_h=filter_h, filter_w=filter_w, stride=stride, pad=pad)
print('===== col =====\n', col)
print('=====')
```

```
...
===== input_data =====
[[[53. 31. 57. 67.]
  [33. 64. 95. 13.]
  [43. 66. 72. 37.]
  [88. 60. 98. 15.]]]

[[81. 27. 29. 17.]
 [ 0. 64. 38. 10.]
 [ 5. 59. 24. 21.]
 [10. 84. 24.  2.]]]
=====
===== col =====
[[53. 31. 57. 33. 64. 95. 43. 66. 72.]
 [31. 57. 67. 64. 95. 13. 66. 72. 37.]
 [33. 64. 95. 43. 66. 72. 88. 60. 98.]
 [64. 95. 13. 66. 72. 37. 60. 98. 15.]
 [81. 27. 29.  0. 64. 38.  5. 59. 24.]
 [27. 29. 17. 64. 38. 10. 59. 24. 21.]
 [ 0. 64. 38.  5. 59. 24. 10. 84. 24.]
 [64. 38. 10. 59. 24. 21. 84. 24.  2.]]
=====
```


- ・プーリング層・・・縦・横方向の空間を小さくする演算
- ・プーリング層の特徴
 - 学習するパラメタがない。
 - チャンネル数は変化しない。
 - 微小な位置変化に対してロバスト（頑健）

< 実践演習：2_6_simple_convolution_network_after.ipynb >

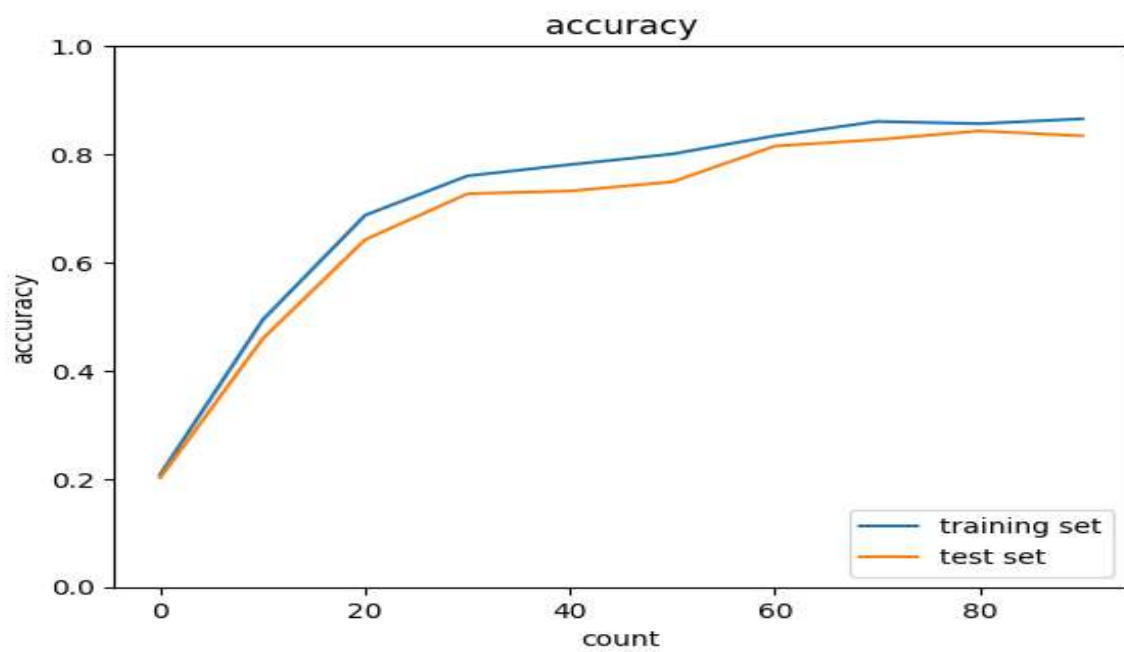


- ・im2col (image to column)

畳み込みニューラルネットワークにて使用される、畳み込み演算の効率化テクニック

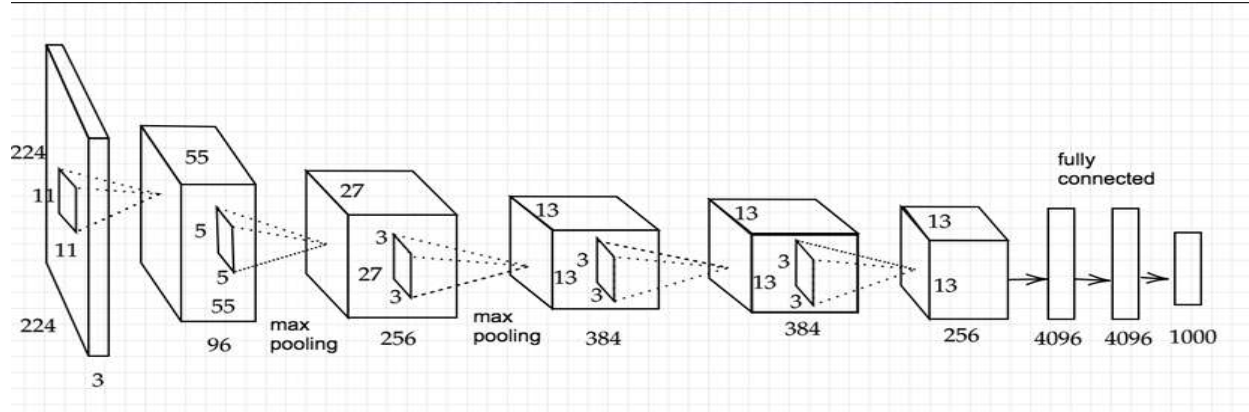
- ・畳み込み演算を行列の積に変換することで、計算処理を高速化
 - 入力特徴マップ上で、畳み込みフィルタがカバーする領域を取得（パディングやストライドの値を基に、スライディングウィンドウで特徴マップの左上から右下までフィルタを移動する）
 - フィルタがカバーする各領域を1次元の列に展開し、全てを並べて行列を作成
 - 畳み込みフィルタも1次元の列に展開し、上記行列との積を計算→結果を再配置して出力特徴マップを生成
- ・メリット：畳み込み演算の行列演算への変換により、最適化された行列乗算アルゴリズムを利用可能→計算効率向上
- ・デメリット：メモリ使用量が増加（重複した値が行列に含まれるため）。特徴マップや畳み込みフィルタのサイズが大きい場合、特に課題となる

< 実践演習 : 2_7_double_convolution_network_after.ipynb >



第五章 最新のCNN

- ・ AlexNet・・・画像を見て「これは何か」を当てるのが得意なAIモデル
- ・ モデルの構造・・・5層の畳み込み層及びプーリング層など、それに続く3層の全結合層から構成される。

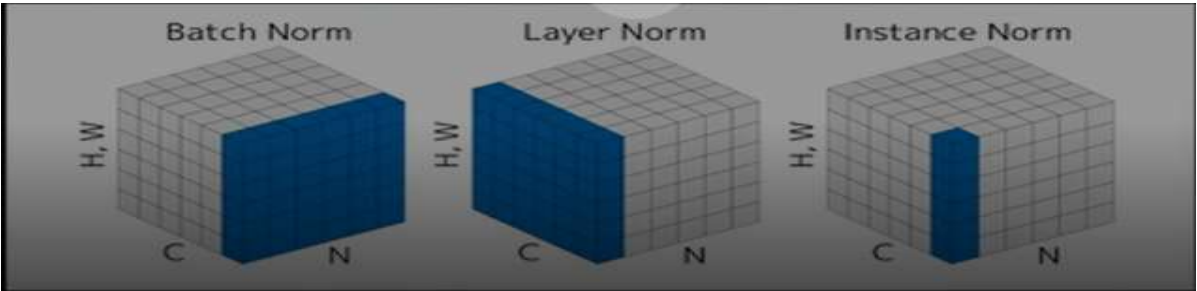


第六章 [フレームワーク演習] 正則化/最適化

- ・ 過学習を抑制する方法

(パラメータ)正則化	正則化レイヤー	正規化レイヤー
L1正則化	Dropout	Batch正規化
L2正則化		Layer正規化
Elastic Net		Instance正規化

正規化レイヤー・・・データの正規化(Normalization)。レイヤー間を流れるデータの分布を何らかの単位で平均が0、分散が1になるように正規化する。



- ・ 正規化レイヤー – Batch正規化
H x W x CのsampleがN個あった場合に、N個の同一チャンネルが正規化の単位
ミニバッチのサイズを大きく取れない場合には、効果が薄くなってしまう
- ・ 正規化レイヤー – Layer正規化
N個のsampleのうち一つに注目。H x W x Cの全てのpixelが正規化の単位。
ミニバッチの数に依存しないので、Batch正規化の問題を解消できていると考えられる。

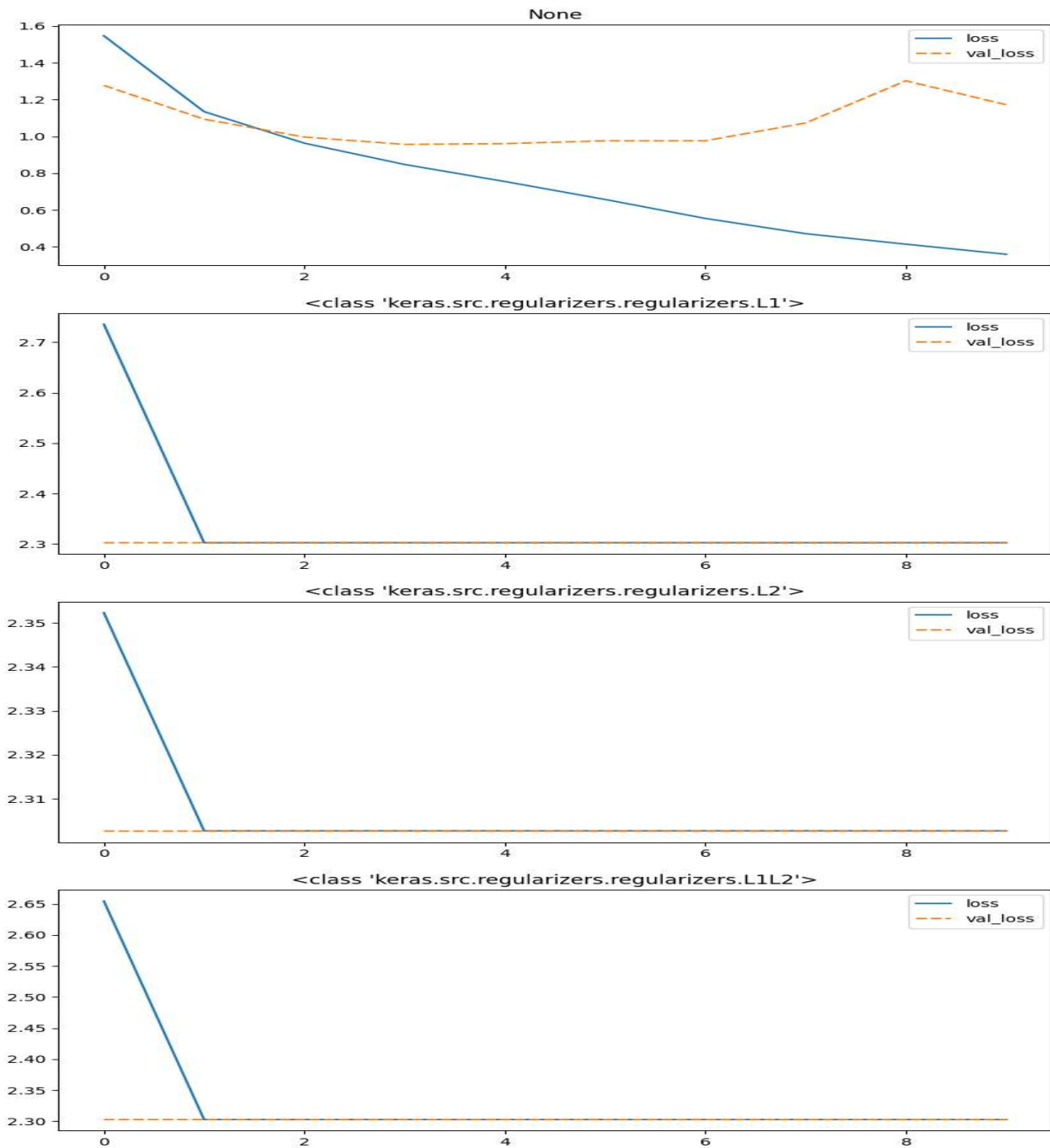
・正規化レイヤー – Instance正規化

各サンプルの各チャンネルごとに正規化。

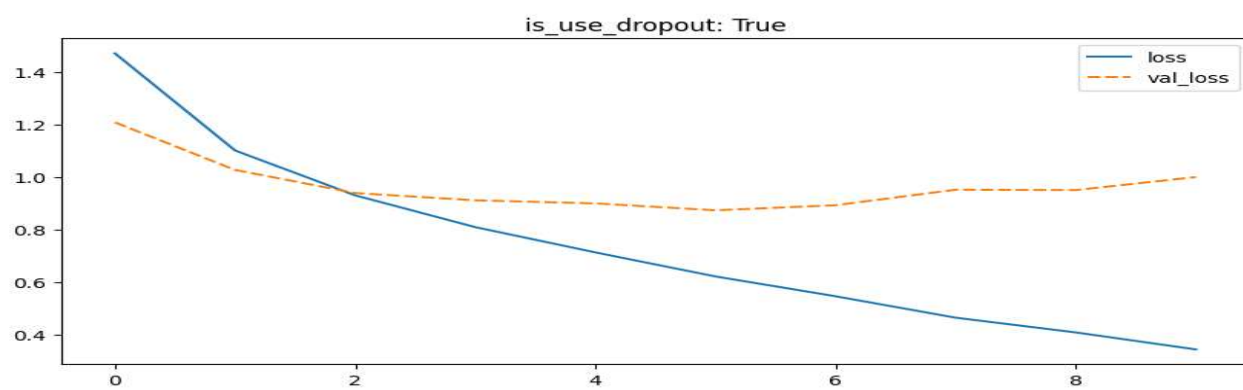
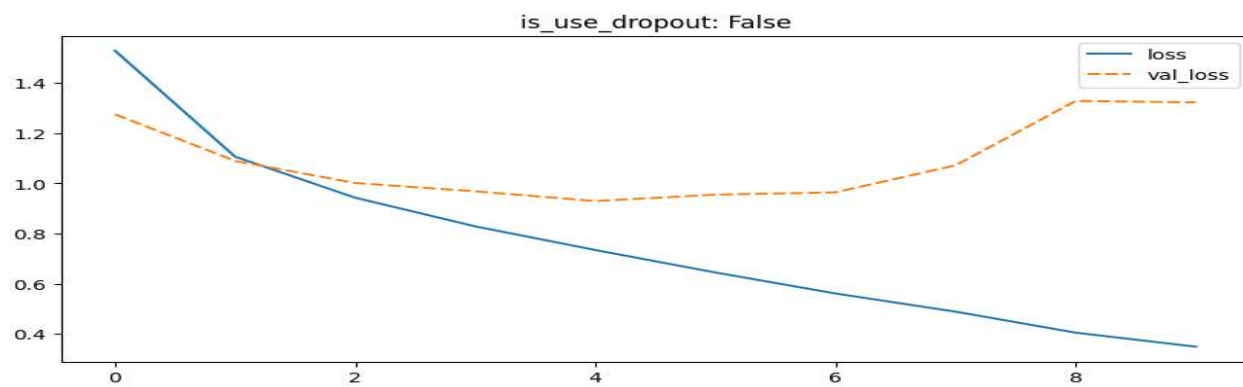
Batch正規化の際のバッチサイズが1場合と等価。

< 実践演習：2_9_regularization.ipynb >

各正則化手法の比較 – L1、L2、ElastiNetをそれぞれ適用した場合の比較

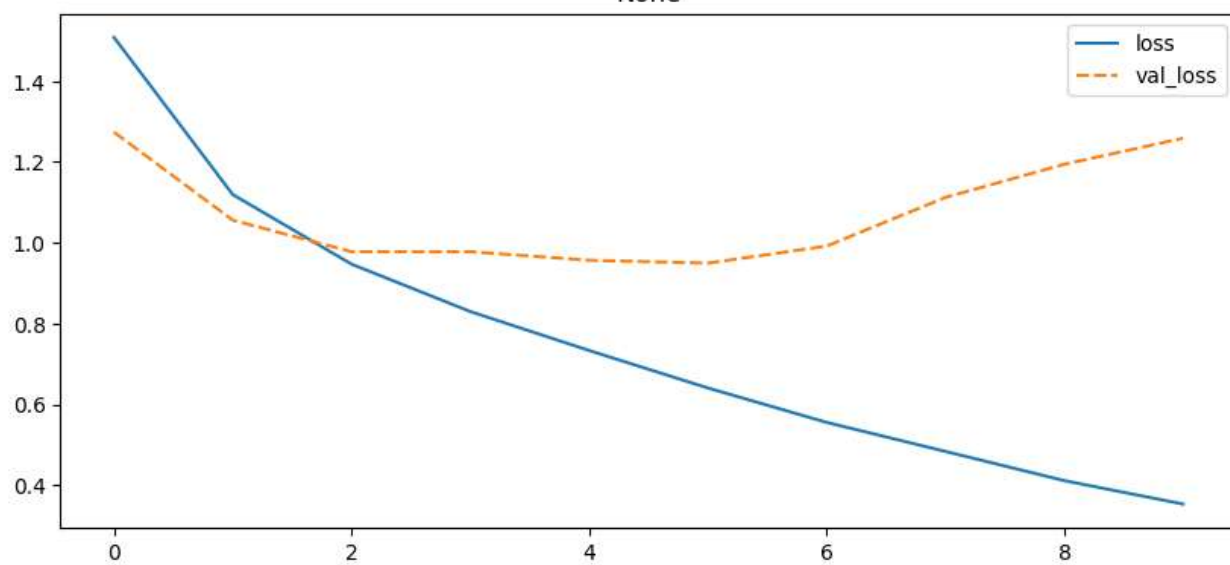


< 実践演習 : 2_10_layer-normalization.ipynb >

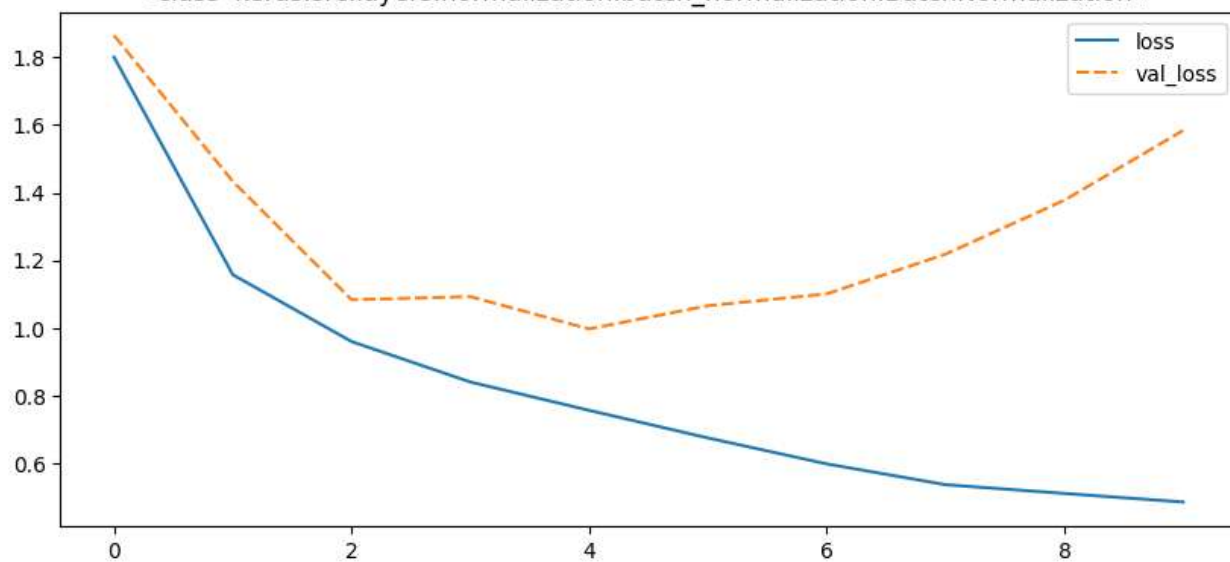


< 実践演習 : 2_10_layer-normalization.ipynb >

None



<class 'keras.src.layers.normalization.batch_normalization.BatchNormalization'>



<class 'keras.src.layers.normalization.layer_normalization.LayerNormalization'>

